

Assessment: Weekly Assessment 3

Info 526 – Data Analysis & Visualization

Student Name: Muhammad Kamran Syed

Contents

[Plot1](#) 2

[Plot 2](#)..... 3

Solution Overview:..... 4

 Data Loading 4

 Cleaning and Transforming the Data: 4

 Preparing Data for answering the following Questions: 6

 Plotting the Data 7

Code..... 11



Figure 01. Above set of plots showing trends of Snail and Slugs over seasons, and months. Below set of plots shows the trend over the years for different species.

The top row of plots displays the Total Observations of Slugs and Snails across different seasons and months.

- The top left plot illustrates the seasonal distribution of slug and snail observations, with total counts plotted for each season (Spring, Summer, Fall, Winter) using separate lines for slugs and snails.
 - The plot shows that the highest total counts for Snail were recorded during Winter particularly in November.
 - For Slugs, the highest observations occurred during Summer, with a peak in June.
 - Overall, the best season for both Slugs and Snails combined was during Summer, with June being the most favorable month for observation of both types. However, due to missing data for many other months, and the positive skewness in the snail data, particularly from November 2006, the significance of this study is limited.

The two plots at the bottom row display the yearly display of Slug and Snail species at more granular level.

- The bottom two plots present the yearly distribution of different species of slugs and snails. Total counts are plotted for each year, with separate panels for slugs and snails, and individual species groups represented by different lines. This comparison allows for a clear visualization of how observations vary across Seasons, Months, and Years, and provides insight into the long-term trends for each species group.



Figure 02. Above set of plots demonstrating Scaling and Ebbinghaus illusions in trends of Snail and Slugs data

The two sets of plots depicted above present the same information as the plots on the previous page. However, we have manually introduced **visual illusions** into these plots, which may lead to misinterpretation of the data:

- The **plots in the top row** demonstrate the **Scaling Illusion**:
 - In the **top-left plot**, the **Y-axis** has been scaled to start from **200** instead of 0, exaggerating differences in total counts and potentially distorting the perceived scale of the observations.
 - In the **top-right plot**, the **months** have been scaled to start from **May** instead of January. This adjustment shifts the x-axis, creating the illusion that the timeline begins at the leftmost point, which could mislead the viewer by altering the perceived flow of observations.
- The **bottom two plots** illustrate the **Ebbinghaus Illusion** through the manipulation of **marker sizes**:
 - We have randomly fluctuated the marker sizes within the plots. While the underlying data and trends remain the same, the varying marker sizes create a visual inconsistency, leading to potential misinterpretation of the significance or frequency of certain observations.

These illusions demonstrate how visual distortions can impact the interpretation of otherwise accurate data, emphasizing the importance of careful scaling and consistent graphical representation in data visualizations.

Solution Overview:

Data Loading

The first step in the notebook is to load the slug and snail dataset. The data contains 12 columns and 3587 rows. We use **Pandas** to import the dataset, clean it, and restructure it for further analysis. This includes:

- Import required package and load the data.

```
# Import packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.dates import DateFormatter
import seaborn as sns
```

```
df_source = pd.read_csv('Slugs+and+Snails.csv')

df = df_source.copy()
df.shape
```

```
df.columns
```

```
df.head()
```

Cleaning and Transforming the Data:

Now we will clean and transform the data, such as:

- First, we normalized the column names to follow same pattern:

```
df.rename(columns={
    'Project.Name': 'ProjectName',
    'UNIT': 'Unit',
    'SEARCH.AREA': 'SearchArea',
    'EASTING': 'Easting',
    'NORTHING': "Northing"
}, inplace=True
)

df.columns
```

- Next, we checked for any duplicates and remove if found:

```
# check for duplicate rows in dataframe
df.duplicated().sum()
```

- Convert Time and Date Columns to Date Time as there are no nulls in these columns. Plus extracted different information from Data which'll be of our user later during data preparation for plotting.

```
df['Time'] = pd.to_datetime(df['Time'], format='%Y-%m-%d %H:%M:%S')
df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m-%d')

# Add column for Year and Month for better Duration Trends
df['Year'] = df['Date'].dt.year
```

```
df['Month'] = df['Date'].dt.strftime('%b')
df['Month Year'] = df['Date'].dt.strftime('%b %Y')
df['Month Year No'] = df['Date'].dt.to_period('M').dt.to_timestamp()

df.sort_values(by=['Date'], inplace=True)

print(df['Year'].unique())
print(df['Month'].unique())
print(df['Month Year'].unique())
print(df['Month Year No'].unique())
```

- Next, we processed the data to include season information in our data set:

```
def get_season_name(date):
    # Dec, Jan, Feb
    if date.month in [12, 1, 2]: return 'Winter'
    # March, April, May
    elif date.month in [3, 4, 5]: return 'Spring'
    # June, July, August
    elif date.month in [6, 7, 8]: return 'Summer'
    # Sep, Oct, Nov
    elif date.month in [9, 10, 11]: return 'Fall'
```

```
df['Season'] = df['Date'].apply(get_season_name)

# Check the unique seasons and their counts
season_counts = df['Season'].value_counts()

print(df['Season'].unique())
print(season_counts)
```

- Remove the rows where Count of observation is missing, or Count is 0 or less.

```
df[(df['Count'].isna()) | (df['Count'] <= 0)].count()

# Drop the 45 rows where count is missing or 0 or lower
df.drop(df[df['Count'] == 0].index, inplace=True)
df.reset_index(drop=True, inplace=True)

df.info()
```

- Convert data types. We converted Count and Year to int32 for better memory usage.

```
# Memory efficient int32
df['Count'] = df['Count'].astype('int32')
df['Year'] = df['Year'].astype('int32')

df.info()
```

- We processed and observed that there are some NAN entries in Easting and Northing, but these will not impact our processing hence we leave it unchanged:
- We also noticed some inconsistencies in Unit data, but since this is not required in our EDA, we left them unchanged

```
df[(df['Easting'].isna()) |
    (df['Northing'].isna())][['Easting', 'Northing',
    'Count']].reset_index(drop=True)
```

```
df[(df['Easting'].isin([467011, 465608])) |
    (df['Northing'] == 5030805)][['Easting', 'Northing',
    'Count']].reset_index(drop=True)
```

```
df.describe(include='all').T
```

- We are only interested in Slug and Snail data, so we filtered the Data Frame for these values.
- We are only interested in some specific columns hence we extracted those, sort the values and saved the resulting dataset to a new file.

```
df_filtered_species = df[(df['Type'].isin(['Slug', 'Snail']))] \
    .reset_index(drop=True)[['Date', 'Year', 'Month Year', 'Month', 'Month Year No',
    'Season', 'Type', 'ScientificName', 'Count']] \
    .sort_values(by=['Date', 'Season', 'Type', 'ScientificName', 'Count']) \
    .reset_index(drop=True)
```

```
df_filtered_species.to_csv('Slug_Snail_Cleaned.csv', index=False)
```

```
df_filtered_species.head()
```

Preparing Data for answering the following Questions:

- . When is the best time of year to find slugs and snails
- . How do the number of slugs and snails found by biologists change over time in this data set?
- . How do the number of slug and snail species found by biologists change over time in this data set?

```
df_filtered_species.sort_values(by='Date', inplace=True)
```

```
df_filtered_species.columns
```

- There are a lot of species hence we define and applied a function for grouping data from similar species.

```
# Define a function that uses if conditions to categorize species into three groups
def categorize_species(scientific_name):
    if scientific_name.startswith('Ancotrema'):
        return 'Ancotrema'
    elif scientific_name.startswith('Prophysaon'):
        return 'Prophysaon'
    elif scientific_name.startswith('Pristiloma'):
        return 'Pristiloma'
    # Unknown species
    elif 'Unknown' in scientific_name or 'Known' in scientific_name:
        return 'Other Species'
    else:
        return scientific_name
```

```
# Step 1: Apply the function to create a new column in the original DataFrame
df_filtered_species['Species_Group'] =
df_filtered_species['ScientificName'].apply(categorize_species)

# species_totals
```

```
df_filtered_species['Species_Group'].unique()
```

- Prepare the data for the plot which shows Season wise Total Count for Snail and Slugs

```
seasonal_type_summary = df_filtered_species.groupby(  
    by=['Season', 'Type']  
) .agg(  
    Total_Count = ('Count', 'sum')  
) .reset_index()  
  
seasonal_type_summary
```

- Prepare the data for the plot which shows month wise Total Count for Snail and Slugs and sort it based on month order.

```
order_of_months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',  
                  'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']  
  
month_type_summary = df_filtered_species.groupby(  
    by=['Month', 'Type']  
) .agg(  
    Total_Count = ('Count', 'sum')  
) .reset_index().sort_values(  
    'Month',  
    key=lambda x: x.map({month: i for i, month in enumerate(order_of_months)}))  
)
```

- Prepare the data for Individual species plot by year by grouping on Year, Type and Species Group and took sum of the count for each group.

```
type_species_summary = df_filtered_species.groupby(  
    by=['Year', 'Type', 'Species_Group']  
) .agg(  
    Total_Count = ('Count', 'sum')  
) .reset_index()  
  
type_species_summary
```

Plotting the Data

- Set universal plotting options like Fonts, Colors etc.

```
# Set global font settings with rcParams for plots  
plt.rcParams['font.family'] = 'Serif'      # Set font family  
plt.rcParams['font.size'] = 12             # Set general font size  
plt.rcParams['axes.labelsize'] = 14        # Font size for axis labels  
plt.rcParams['axes.labelcolor'] = 'purple'  
plt.rcParams['axes.titlesize'] = 18        # Font size for the title  
plt.rcParams['axes.titlepad'] = 20.0  
plt.rcParams['axes.labelpad'] = 15.0  
plt.rcParams['axes.titlecolor'] = 'purple'  
plt.rcParams['xtick.color'] = 'navy'  
plt.rcParams['ytick.color'] = 'navy'  
plt.rcParams['legend.fontsize'] = 12      # Font size for legend  
plt.rcParams['legend.frameon'] = True
```

- Define the month order for x labels on the the plot and convert the Month column datatype to Categories based on the month order.

```
# Define the correct order of months
month_order = ['Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
               'Sep', 'Oct', 'Nov', 'Dec', 'Jan', 'Feb']

# Convert 'Month' to a categorical variable with the correct order
month_type_summary['Month'] = pd.Categorical(
    month_type_summary['Month'], categories=month_order, ordered=True
)
```

- Plot the 2 Line plots side by side using subplots, showing:
 - Season wise Snail and Slug Total Count
 - Month wise Snail and Slug Total Count

```
# Set up the figure for subplots (2 rows, 1 column)
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(16, 6))

# Plot 1: Line plot for Total Observations per Season
sns.lineplot(data=seasonal_type_summary, x='Season', y='Total_Count', hue='Type',
             marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.7,
             ax=ax1)

season_order = seasonal_type_summary['Season'].unique()

# Customize ax1
ax1.set_xlabel('Season', fontsize=12)
ax1.set_ylabel('Total Count', fontsize=12)
ax1.set_xticks(range(len(season_order))) # Set tick positions
ax1.set_xticklabels(season_order, rotation=15)

# Plot 2: Line plot for Total Observations per Month
sns.lineplot(data=month_type_summary, x='Month', y='Total_Count', hue='Type',
             marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.7,
             ax=ax2)

# Customize ax2
ax2.set_xlabel('Month', fontsize=12)
ax2.set_ylabel('Total Count', fontsize=12)
#ax2.set_title('Total Observations of Slug and Snail per Month', fontsize=14)
ax2.set_xticks(range(len(month_order)))
ax2.set_xticklabels(month_order, rotation=15)

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the combined plots
plt.show()
```

- Plot the 2 Scaling Illusional Line plots side by side showing:
 - Season wise Snail and Slug Total Count – Scaling Illusion on Y axis
 - Month wise Snail and Slug Total Count – Scaling Illusion on X axis

```
# Set up the figure for subplots (2 columns, 1 row)
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(16, 6))

#Change the order of Seasons on X Axis
new_order = ['Winter', 'Spring', 'Summer', 'Fall']
seasonal_type_summary['Season'] = pd.Categorical(
    seasonal_type_summary['Season'], categories=new_order, ordered=True
)

# Plot 1: Line plot for Total Observations per Season with scale illusion
sns.lineplot(data=seasonal_type_summary, x='Season', y='Total_Count', hue='Type',
             marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.7,
             ax=ax1)

# Customize ax1 with scale illusion (start y-axis at 500 instead of 0)
ax1.set_ylim(200, seasonal_type_summary['Total_Count'].max() + 100) # Y-axis doesn't
start at 0
ax1.set_xlabel('Season', fontsize=12)
ax1.set_ylabel('Total Count', fontsize=12)
#ax1.set_title('Total Observations of Slug and Snail per Season (with Scale
Illusion)', fontsize=14)

#ax1.set_xticks(range(len(new_order))) # Set tick positions
ax1.set_xticklabels(season_order, rotation=15)

# Plot 2: Line plot for Total Observations per Month with scale illusion
sns.lineplot(data=month_type_summary, x='Month', y='Total_Count', hue='Type',
             marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.7,
             ax=ax2)

# Customize ax2 with scale illusion (start y-axis at 300 instead of 0)
# X axis are missing some of the months where no observations were recorded
ax2.set_ylim(200, month_type_summary['Total_Count'].max() + 100) # Y-axis doesn't
start at 0
ax2.set_xlabel('Month', fontsize=12)
ax2.set_ylabel('Total Count', fontsize=12)
#ax2.set_title('Total Observations of Slug and Snail per Month (with Scale Illusion)',
fontsize=14)

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the combined plots
plt.show()
```

- Plot the 2 plots side by side using Facet Grid for:
 - Granular Species Trend over the years for Slugs
 - Granular Species Trend over the years for Snails
 - Exclude any low number (below 5) for avoiding clutter

```
type_species_summary[type_species_summary.Total_Count <= 5]
```

```
# Adjust overall figure size to make the plots larger
plt.figure(figsize=(14, 12)) # Increase figure size to make it more readable

# Create a FacetGrid to show Slugs and Snails in separate facets, exclude minor
entries to reduce cluttering
g = sns.FacetGrid(data=type_species_summary[type_species_summary.Total_Count >= 10],
                  col='Type', height=6, aspect=0.8)

# Map the line plot onto the FacetGrid, with separate lines for each Species_Group
g.map_dataframe(sns.lineplot, x='Year', y='Total_Count', hue='Species_Group',
               marker='o',
               markersize=10, linewidth=3, alpha=0.8)

# Add a legend for the Species_Group and move it outside the plot area
g.add_legend(title="Species", bbox_to_anchor=(1.22, 0.97), loc='upper right')

# Adjust axis labels and titles
g.set_axis_labels("Year", "Total Count", color='purple')
g.set_titles("{col_name}") # Shows Slug or Snail in the column titles

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the plot
plt.show()
```

- Plot the 2 plots side by side containing Ebbinghaus Illusion on Facet Grid for:
 - Granular Species Trend for Slugs
 - Granular Species Trend over the years for Snails
 - Exclude any low number (below 5) for avoiding clutter

```
# Adjust overall figure size to make the plots larger
plt.figure(figsize=(14, 12)) # Increase figure size to make it more readable

# Introduce an Ebbinghaus-like illusion by adjusting marker sizes based on a pattern
type_species_summary['Marker_Size'] = np.random.randint(50, 301,
size=len(type_species_summary))
#type_species_summary['Marker_Size'] = np.where(type_species_summary['Year'] % 2 == 1,
200, 20)

# Create a FacetGrid to show Slugs and Snails in separate facets
g = sns.FacetGrid(data=type_species_summary[type_species_summary.Total_Count >= 10],
                  col='Type', height=6, aspect=0.8)

# Map the line plot onto the FacetGrid, with separate lines for each Species_Group and
marker sizes
g.map_dataframe(sns.lineplot, x='Year', y='Total_Count', hue='Species_Group',
               marker='o', linewidth=3, alpha=0.8)
g.map_dataframe(sns.scatterplot, x='Year', y='Total_Count', hue='Species_Group',
```

```
size='Marker_Size', sizes=(50, 300), legend=False, alpha=0.8)

# Add a legend for the Species_Group and move it outside the plot area
g.add_legend(title="Species", bbox_to_anchor=(1.22, 0.97), loc='upper right')

# Adjust axis labels and titles
g.set_axis_labels("Year", "Total Count", color='purple')
g.set_titles("{col_name}") # Shows Slug or Snail in the column titles

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the plot
plt.show()
```

There are some other variations of plot as well which can be found in the `assessment_3_solution.ipynb` file submitted as a solution.

Code

Refer to the next page for all the codes (ipynb notebook as attachment):

The following versions were used for this code

- Python 3.12.6
- Pandas 2.2.2
- Numpy 2.1.1
- Matplotlib 3.9.2
- Seaborn 0.13.2

Python version 3.12.6 was used for this code

Use `! pip install package_name` to install any missing packages

assessment-3-solution

October 8, 2024

1 Weekly Assessment 3

1.0.1 INFO 526 - Data Analysis & Visualization

1.0.2 Student Name: Muhammad Kamran Syed

1.0.3 Problem Statement:

Create a Pretty Plot and Illusion Plot for answering the below questions:

1- When is the best time of year to find slugs and snails? (Slugs and Snails Dataset)

Use the slugs and snails data set to create a plot that answers the following questions. Slugs and snails are very temperature and moisture dependent. They hibernate during the winter when it is too cold and aestivate (a type of summer hibernation) when it is too hot in the summer. Surveys are conducted for them in the spring and the fall, so you will only be able to evaluate within those two time periods.

How do the number of slugs and snails found by biologists change over time in this data set? How do the number of slug and snail species found by biologists change over time in this data set?

```
[139]: # Import packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.dates import DateFormatter
import seaborn as sns
```

1.1 Step 1: Understand the data

Lets load the data from 'Slugs+and+Snails.csv' file and examine it.

```
[140]: df_source = pd.read_csv('Slugs+and+Snails.csv')

df = df_source.copy()
df.shape
```

```
[140]: (3587, 12)
```

1.1.1 Standardize the column names

```
[141]: df.columns
```

```
[141]: Index(['Project.Name', 'UNIT', 'SEARCH.AREA', 'Time', 'Date', 'EASTING',  
          'NORTHING', 'MolluskCode', 'ScientificName', 'Type', 'CommonName',  
          'Count'],  
          dtype='object')
```

```
[142]: df.head()
```

```
[142]:
```

	Project.Name	UNIT	SEARCH.AREA	Time	Date	EASTING	\
0	Nehalem	31-1	SA1	2011-10-28 15:00:00	2011-10-28	492073.0	
1	Nehalem	31-1	SA10	2011-10-28 16:45:00	2011-10-28	492064.0	
2	Nehalem	31-1	SA10	2011-10-28 16:45:00	2011-10-28	492064.0	
3	Nehalem	31-1	SA12	2011-10-27 17:45:00	2011-10-27	492186.0	
4	Nehalem	31-1	SA13	2011-10-28 08:00:00	2011-10-28	492265.0	

	NORTHING	MolluskCode	ScientificName	Type	\
0	5080757.0	VECO	Vespericola columbianus	Snail	
1	5080488.0	ARCO	Ariolimax columbiana	Slug	
2	5080488.0	VECO	Vespericola columbianus	Snail	
3	5080244.0	ANSP	Ancotrema sportella	Snail	
4	508029.0	HAVA	Haplotrema vancouverense	Snail	

	CommonName	Count
0	Northwest Hesperian snail	2
1	Banana slug	1
2	Northwest Hesperian snail	2
3	Beaded lancetooth snail	1
4	Robust lancetooth snail	3

```
[143]: df.rename(columns={  
          'Project.Name': 'ProjectName',  
          'UNIT': 'Unit',  
          'SEARCH.AREA': 'SearchArea',  
          'EASTING': 'Easting',  
          'NORTHING': 'Northing',  
        }, inplace=True  
      )  
  
df.columns
```

```
[143]: Index(['ProjectName', 'Unit', 'SearchArea', 'Time', 'Date', 'Easting',  
          'Northing', 'MolluskCode', 'ScientificName', 'Type', 'CommonName',  
          'Count'],  
          dtype='object')
```

1.1.2 Convert DateTime Columns

```
[144]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3587 entries, 0 to 3586
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   ProjectName     3587 non-null   object
1   Unit            3587 non-null   object
2   SearchArea      3587 non-null   object
3   Time            3587 non-null   object
4   Date            3587 non-null   object
5   Easting         3572 non-null   float64
6   Northing        3570 non-null   float64
7   MolluskCode     3587 non-null   object
8   ScientificName  3587 non-null   object
9   Type            3587 non-null   object
10  CommonName      3587 non-null   object
11  Count           3587 non-null   int64
dtypes: float64(2), int64(1), object(9)
memory usage: 336.4+ KB
```

```
[145]: # check for duplicate rows in dataframe
df.duplicated().sum()
```

```
[145]: 0
```

1.1.3 Convert Time and Date Columns to DateTime as there are no nulls in these columns

```
[146]: df['Time'] = pd.to_datetime(df['Time'], format='%Y-%m-%d %H:%M:%S')
df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m-%d')

# Add column for Year and Month for better Duration Trends
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.strftime('%b')
df['Month Year'] = df['Date'].dt.strftime('%b %Y')
df['Month Year No'] = df['Date'].dt.to_period('M').dt.to_timestamp()

df.sort_values(by=['Date'], inplace=True)

print(df['Year'].unique())
print(df['Month'].unique())
print(df['Month Year'].unique())
print(df['Month Year No'].unique())
```

```
[2006 2011 2012]
['May' 'Jun' 'Nov' 'Oct' 'Dec']
['May 2006' 'Jun 2006' 'Nov 2006' 'Jun 2011' 'Oct 2011' 'Nov 2011'
 'Dec 2011' 'May 2012' 'Jun 2012' 'Nov 2012']
<DatetimeArray>
['2006-05-01 00:00:00', '2006-06-01 00:00:00', '2006-11-01 00:00:00',
 '2011-06-01 00:00:00', '2011-10-01 00:00:00', '2011-11-01 00:00:00',
 '2011-12-01 00:00:00', '2012-05-01 00:00:00', '2012-06-01 00:00:00',
 '2012-11-01 00:00:00']
Length: 10, dtype: datetime64[ns]
```

1.1.4 Get the name of seasons From Date

Winter = Dec, Jan, Feb
 Spring = Mar, Apr, May
 Summer = Jun, Jul, Aug
 Fall = Sep, Oct, Nov

```
[147]: def get_season_name(date):
        # Dec, Jan, Feb
        if date.month in [12, 1, 2]: return 'Winter'
        # March, April, May
        elif date.month in [3, 4, 5]: return 'Spring'
        # June, July, August
        elif date.month in [6, 7, 8]: return 'Summer'
        # Sep, Oct, Nov
        elif date.month in [9, 10, 11]: return 'Fall'
```

```
[148]: df['Season'] = df['Date'].apply(get_season_name)

        # Check the unique seasons and their counts
        season_counts = df['Season'].value_counts()

        print(df['Season'].unique())
        print(season_counts)
```

```
['Spring' 'Summer' 'Fall' 'Winter']
Season
Fall      1417
Summer    1284
Spring     836
Winter      50
Name: count, dtype: int64
```

```
[149]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3587 entries, 292 to 3125
```

Data columns (total 17 columns):

#	Column	Non-Null Count	Dtype
0	ProjectName	3587 non-null	object
1	Unit	3587 non-null	object
2	SearchArea	3587 non-null	object
3	Time	3587 non-null	datetime64[ns]
4	Date	3587 non-null	datetime64[ns]
5	Easting	3572 non-null	float64
6	Northing	3570 non-null	float64
7	MolluskCode	3587 non-null	object
8	ScientificName	3587 non-null	object
9	Type	3587 non-null	object
10	CommonName	3587 non-null	object
11	Count	3587 non-null	int64
12	Year	3587 non-null	int32
13	Month	3587 non-null	object
14	Month Year	3587 non-null	object
15	Month Year No	3587 non-null	datetime64[ns]
16	Season	3587 non-null	object

dtypes: datetime64[ns](3), float64(2), int32(1), int64(1), object(10)

memory usage: 490.4+ KB

1.1.5 Lets check the count columns

```
[150]: df['Count'].unique()
```

```
[150]: array([ 1,  2,  3,  4, 11, 15,  5,  7, 17,  8, 13, 12,  0,  6,  9, 10, 14,
        16, 23], dtype=int64)
```

```
[151]: df[(df['Count'].isna()) | (df['Count'] <= 0)].count()
```

```
[151]: ProjectName      45
      Unit             45
      SearchArea       45
      Time             45
      Date             45
      Easting          45
      Northing         42
      MolluskCode       45
      ScientificName    45
      Type             45
      CommonName       45
      Count            45
      Year             45
      Month            45
      Month Year        45
```

```
Month Year No      45
Season          45
dtype: int64
```

1.1.6 We can drop these 45 rows where count is 0 as this will not be utilized

```
[152]: # Drop the 45 rows where count is missing or 0 or lower
df.drop(df[df['Count'] == 0].index, inplace=True)
df.reset_index(drop=True, inplace=True)

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3542 entries, 0 to 3541
Data columns (total 17 columns):
#   Column          Non-Null Count  Dtype
---  -
0   ProjectName     3542 non-null   object
1   Unit            3542 non-null   object
2   SearchArea      3542 non-null   object
3   Time            3542 non-null   datetime64[ns]
4   Date            3542 non-null   datetime64[ns]
5   Easting         3527 non-null   float64
6   Northing        3528 non-null   float64
7   MolluskCode     3542 non-null   object
8   ScientificName  3542 non-null   object
9   Type            3542 non-null   object
10  CommonName      3542 non-null   object
11  Count           3542 non-null   int64
12  Year            3542 non-null   int32
13  Month           3542 non-null   object
14  Month Year       3542 non-null   object
15  Month Year No    3542 non-null   datetime64[ns]
16  Season          3542 non-null   object
dtypes: datetime64[ns](3), float64(2), int32(1), int64(1), object(10)
memory usage: 456.7+ KB
```

1.1.7 Convert Datatypes to Integer

We will convert Count and Year to int32 for better memory usage.

```
[153]: # Memory efficient int32
df['Count'] = df['Count'].astype('int32')
df['Year'] = df['Year'].astype('int32')

df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3542 entries, 0 to 3541
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ProjectName           3542 non-null   object
1   Unit                  3542 non-null   object
2   SearchArea            3542 non-null   object
3   Time                  3542 non-null   datetime64[ns]
4   Date                  3542 non-null   datetime64[ns]
5   Easting               3527 non-null   float64
6   Northing              3528 non-null   float64
7   MolluskCode           3542 non-null   object
8   ScientificName         3542 non-null   object
9   Type                  3542 non-null   object
10  CommonName            3542 non-null   object
11  Count                 3542 non-null   int32
12  Year                  3542 non-null   int32
13  Month                 3542 non-null   object
14  Month Year            3542 non-null   object
15  Month Year No         3542 non-null   datetime64[ns]
16  Season                3542 non-null   object
dtypes: datetime64[ns](3), float64(2), int32(2), object(10)
memory usage: 442.9+ KB

```

1.1.8 Lack of information & Issues in Data

We can also observe that there is no such info available which we can interchange or use for forward or backward filling. However, since there are Count available for these observations, we can't just drop them.

```

[154]: df[(df['Easting'].isna()) |
         (df['Northing'].isna())][['Easting', 'Northing', 'Count']].
         ↪reset_index(drop=True)

```

```

[154]:
   Easting  Northing  Count
0      NaN      NaN      2
1      NaN      NaN      2
2      NaN      NaN      3
3      NaN      NaN      1
4      NaN      NaN      4
5      NaN      NaN      1
6      NaN      NaN      1
7      NaN      NaN      2
8      NaN      NaN      1
9      NaN      NaN      5
10     NaN  5030805.0      1
11     NaN  5030805.0      4

```

12	NaN	5030805.0	15
13	NaN	5030805.0	2
14	NaN	5030805.0	3
15	465608.0	NaN	1
16	465608.0	NaN	2
17	467011.0	NaN	2
18	467011.0	NaN	1

```
[155]: df[(df['Easting'].isin([467011, 465608])) |
          (df['Northing'] == 5030805)][['Easting', 'Northing', 'Count']].
        ↪reset_index(drop=True)
```

```
[155]:
```

	Easting	Northing	Count
0	NaN	5030805.0	1
1	NaN	5030805.0	4
2	NaN	5030805.0	15
3	NaN	5030805.0	2
4	NaN	5030805.0	3
5	465608.0	NaN	1
6	465608.0	NaN	2
7	467011.0	NaN	2
8	467011.0	NaN	1

```
[156]: df.shape
```

```
[156]: (3542, 17)
```

1.1.9 Problem in Data

While, rest of the columns are now properly cleaned, we can see Northing and Easting still have some NAs, However we don't have enough info to fix these columns neither are these important to answer our 3 questions. Hence we will ignore these unresolved missing data.

Similarly we can observe some issues and inconsistencies in the Unit data as well. We will ignore these too.

```
[157]: df.describe(include='all').T
```

```
[157]:
```

	count	unique	top	freq	\
ProjectName	3542	5	Roaring Creek	2147	
Unit	3542	37	2	809	
SearchArea	3542	997	9SA28-1	11	
Time	3542	NaN	NaN	NaN	
Date	3542	NaN	NaN	NaN	
Easting	3527.0	NaN	NaN	NaN	
Northing	3528.0	NaN	NaN	NaN	
MolluskCode	3542	43	HAVA	750	
ScientificName	3542	43	Haplotrema vancouverense	750	

Type	3542	5	Snail	2761
CommonName	3542	42	Robust lancetooth snail	750
Count	3542.0	NaN	NaN	NaN
Year	3542.0	NaN	NaN	NaN
Month	3542	5	Nov	1300
Month Year	3542	10	Nov 2006	1070
Month Year No	3542	NaN	NaN	NaN
Season	3542	4	Fall	1412

		mean	min	\
ProjectName		NaN	NaN	
Unit		NaN	NaN	
SearchArea		NaN	NaN	
Time	2008-09-10 02:09:32.811970816	2006-05-15 12:00:00		
Date	2008-09-09 13:44:53.393562880	2006-05-15 00:00:00		
Easting	486210.178055	46457.0		
Northing	5009124.68339	502679.0		
MolluskCode	NaN	NaN		
ScientificName	NaN	NaN		
Type	NaN	NaN		
CommonName	NaN	NaN		
Count	1.975438	1.0		
Year	2008.084416	2006.0		
Month	NaN	NaN		
Month Year	NaN	NaN		
Month Year No	2008-08-25 10:52:55.155279360	2006-05-01 00:00:00		
Season	NaN	NaN		

		25%	50%	75%	\
ProjectName		NaN	NaN	NaN	
Unit		NaN	NaN	NaN	
SearchArea		NaN	NaN	NaN	
Time	2006-06-03 12:00:00	2006-11-20 12:00:00	2011-06-17 08:00:00		
Date	2006-06-03 00:00:00	2006-11-20 00:00:00	2011-06-17 00:00:00		
Easting	473873.0	475286.0	476600.0		
Northing	5028808.0	5031806.0	5033412.0		
MolluskCode	NaN	NaN	NaN		
ScientificName	NaN	NaN	NaN		
Type	NaN	NaN	NaN		
CommonName	NaN	NaN	NaN		
Count	1.0	1.0	2.0		
Year	2006.0	2006.0	2011.0		
Month	NaN	NaN	NaN		
Month Year	NaN	NaN	NaN		
Month Year No	2006-06-01 00:00:00	2006-11-01 00:00:00	2011-06-01 00:00:00		
Season	NaN	NaN	NaN		

	max	std
ProjectName	NaN	NaN
Unit	NaN	NaN
SearchArea	NaN	NaN
Time	2012-11-29 13:15:00	NaN
Date	2012-11-29 00:00:00	NaN
Easting	5007000.0	214358.398595
Northing	5080813.0	331811.790532
MolluskCode	NaN	NaN
ScientificName	NaN	NaN
Type	NaN	NaN
CommonName	NaN	NaN
Count	23.0	1.790406
Year	2012.0	2.601987
Month	NaN	NaN
Month Year	NaN	NaN
Month Year No	2012-11-01 00:00:00	NaN
Season	NaN	NaN

1.1.10 We are only interested in the following information.

1. Date (Time Series Info)
2. Year (For year wise segregation)
3. Month Year
4. Month
5. Month Year No
6. Season (Spring and Fall)
7. Type (For Species)
8. ScientificName (For granular species)
9. Count (Count of Observations)

1.1.11 While the question suggest that the data is only available for Spring and Fall, we observed that the data is available for other seasons as well. Hence we will include those as well.

```
[158]: df_filtered_species = df[(df['Type'].isin(['Slug', 'Snail']))] \
        .reset_index(drop=True)[['Date', 'Year', 'Month Year', 'Month', 'Month Year_
        ↪No',
                                'Season', 'Type', 'ScientificName', 'Count']] \
        .sort_values(by=['Date', 'Season', 'Type', 'ScientificName', 'Count']) \
        .reset_index(drop=True)

df_filtered_species.to_csv('Slug_Snail_Cleaned.csv', index=False)

df_filtered_species.head()
```

```
[158]:
```

	Date	Year	Month	Year	Month	Month	Year	No	Season	Type	\
0	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
1	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
2	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
3	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
4	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			

	ScientificName	Count
0	Ariolimax columbiana	1
1	Ariolimax columbiana	1
2	Ariolimax columbiana	1
3	Ariolimax columbiana	1
4	Ariolimax columbiana	1

1.1.12 Answering Questions:

- . When is the best time of year to find slugs and snails
- . How do the number of slugs and snails found by biologists change over time in this data set?
- . How do the number of slug and snail species found by biologists change over time in this data set?

```
[159]: df_filtered_species.sort_values(by='Date', inplace=True)
```

```
[160]: df_filtered_species.columns
```

```
[160]: Index(['Date', 'Year', 'Month Year', 'Month', 'Month Year No', 'Season',
            'Type', 'ScientificName', 'Count'],
            dtype='object')
```

```
[161]: # Define a function that uses if conditions to categorize species into three_
        ↪ groups
def categorize_species(scientific_name):
    if scientific_name.startswith('Ancotrema'):
        return 'Ancotrema'
    elif scientific_name.startswith('Prophysaon'):
        return 'Prophysaon'
    elif scientific_name.startswith('Pristiloma'):
        return 'Pristiloma'
    # Unknown species
    elif 'Unknown' in scientific_name or 'Known' in scientific_name:
        return 'Other Species'
    else:
        return scientific_name
```

```
[162]: # Step 1: Apply the function to create a new column in the original DataFrame
df_filtered_species['Species_Group'] = df_filtered_species['ScientificName'].
        ↪ apply(categorize_species)
```

```
# species_totals
df_filtered_species['Species_Group'].unique()
```

```
[162]: array(['Ariolimax columbiana', 'Haplotrema vancouverense',
        'Monadenia fidelis', 'Vertigo columbiana',
        'Vespericola columbianus', 'Prophysaon', 'Ancotrema',
        'Cryptomastix germana', 'Punctum randolphi', 'Pristiloma',
        'Other Species', 'Columella edentula,', 'Hesperarion mariae',
        'Haplotrema sp.', 'Euconulus fulvus', 'Striatura pugetensis',
        'Zacholeus new species', 'Zonitoides arboreus',
        'Hemphillia dromedarius', 'Hemphillia glandulosa',
        'Megomphix hemphilli', 'Allogona townsendiana'], dtype=object)
```

```
[163]: type_species_summary = df_filtered_species.groupby(
        by=['Year', 'Type', 'Species_Group']
    ).agg(
        Total_Count = ('Count', 'sum')
    ).reset_index()

type_species_summary
```

```
[163]:
```

	Year	Type	Species_Group	Total_Count
0	2006	Slug	Ariolimax columbiana	353
1	2006	Slug	Hesperarion mariae	1
2	2006	Slug	Other Species	3
3	2006	Slug	Prophysaon	40
4	2006	Snail	Ancotrema	872
5	2006	Snail	Columella edentula,	128
6	2006	Snail	Cryptomastix germana	96
7	2006	Snail	Euconulus fulvus	16
8	2006	Snail	Haplotrema sp.	2
9	2006	Snail	Haplotrema vancouverense	1702
10	2006	Snail	Monadenia fidelis	162
11	2006	Snail	Other Species	9
12	2006	Snail	Pristiloma	95
13	2006	Snail	Punctum randolphi	127
14	2006	Snail	Striatura pugetensis	13
15	2006	Snail	Vertigo columbiana	152
16	2006	Snail	Vespericola columbianus	796
17	2011	Slug	Ariolimax columbiana	125
18	2011	Slug	Hemphillia dromedarius	11
19	2011	Slug	Hemphillia glandulosa	34
20	2011	Slug	Hesperarion mariae	11
21	2011	Slug	Prophysaon	107
22	2011	Slug	Zacholeus new species	17
23	2011	Snail	Allogona townsendiana	3

24	2011	Snail	Ancotrema	40
25	2011	Snail	Cryptomastix germana	58
26	2011	Snail	Euconulus fulvus	2
27	2011	Snail	Haplotrema sp.	293
28	2011	Snail	Haplotrema vancouverense	335
29	2011	Snail	Megomphix hemphilli	2
30	2011	Snail	Monadenia fidelis	49
31	2011	Snail	Pristiloma	79
32	2011	Snail	Vertigo columbiana	24
33	2011	Snail	Vespericola columbianus	346
34	2011	Snail	Zonitoides arboreus	7
35	2012	Slug	Ariolimax columbiana	65
36	2012	Slug	Hemphillia dromedarius	7
37	2012	Slug	Hemphillia glandulosa	58
38	2012	Slug	Hesperarion mariae	26
39	2012	Slug	Prophysaon	37
40	2012	Slug	Zacholeus new species	11
41	2012	Snail	Ancotrema	10
42	2012	Snail	Cryptomastix germana	19
43	2012	Snail	Haplotrema sp.	36
44	2012	Snail	Haplotrema vancouverense	155
45	2012	Snail	Monadenia fidelis	7
46	2012	Snail	Pristiloma	80
47	2012	Snail	Vertigo columbiana	30
48	2012	Snail	Vespericola columbianus	73
49	2012	Snail	Zonitoides arboreus	4

```
[164]: year_seasonal_type_summary = df_filtered_species.groupby(
        by=['Season', 'Year', 'Type']
    ).agg(
        Total_Count = ('Count', 'sum')
    ).reset_index()

year_seasonal_type_summary
```

```
[164]:
```

	Season	Year	Type	Total_Count
0	Fall	2006	Slug	46
1	Fall	2006	Snail	2468
2	Fall	2011	Slug	99
3	Fall	2011	Snail	392
4	Fall	2012	Slug	6
5	Fall	2012	Snail	2
6	Spring	2006	Slug	231
7	Spring	2006	Snail	1065
8	Spring	2012	Slug	62
9	Spring	2012	Snail	193
10	Summer	2006	Slug	120

11	Summer	2006	Snail	637
12	Summer	2011	Slug	196
13	Summer	2011	Snail	794
14	Summer	2012	Slug	136
15	Summer	2012	Snail	219
16	Winter	2011	Slug	10
17	Winter	2011	Snail	52

```
[165]: seasonal_type_summary = df_filtered_species.groupby(
        by=['Season', 'Type']
    ).agg(
        Total_Count = ('Count', 'sum')
    ).reset_index()

seasonal_type_summary
```

```
[165]:
```

	Season	Type	Total_Count
0	Fall	Slug	151
1	Fall	Snail	2862
2	Spring	Slug	293
3	Spring	Snail	1258
4	Summer	Slug	452
5	Summer	Snail	1650
6	Winter	Slug	10
7	Winter	Snail	52

```
[182]: order_of_months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

month_type_summary = df_filtered_species.groupby(
    by=['Month', 'Type']
).agg(
    Total_Count = ('Count', 'sum')
).reset_index().sort_values(
    'Month',
    key=lambda x: x.map({month: i for i, month in enumerate(order_of_months)})
)

month_type_summary
```

```
[182]:
```

	Month	Type	Total_Count
4	May	Slug	293
5	May	Snail	1258
2	Jun	Slug	452
3	Jun	Snail	1650
8	Oct	Slug	35
9	Oct	Snail	131

6	Nov	Slug	116
7	Nov	Snail	2731
0	Dec	Slug	10
1	Dec	Snail	52

```
[167]: # Set global font settings with rcParams for plots
plt.rcParams['font.family'] = 'Serif' # Set font family
plt.rcParams['font.size'] = 12 # Set general font size
plt.rcParams['axes.labelsize'] = 14 # Font size for axis labels
plt.rcParams['axes.labelcolor'] = 'purple'
plt.rcParams['axes.titlesize'] = 18 # Font size for the title
plt.rcParams['axes.titlepad'] = 20.0
plt.rcParams['axes.labelpad'] = 15.0
plt.rcParams['axes.titlecolor'] = 'purple'
plt.rcParams['xtick.color'] = 'navy'
plt.rcParams['ytick.color'] = 'navy'
plt.rcParams['legend.fontsize'] = 12 # Font size for legend
plt.rcParams['legend.frameon'] = True
```

```
[168]: type_species_summary.columns
```

```
[168]: Index(['Year', 'Type', 'Species_Group', 'Total_Count'], dtype='object')
```

```
[169]: # Define the correct order of months for Plotting
month_order = ['Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
               'Sep', 'Oct', 'Nov', 'Dec', 'Jan', 'Feb']

# Convert 'Month' to a categorical variable with the correct order
month_type_summary['Month'] = pd.Categorical(
    month_type_summary['Month'], categories=month_order, ordered=True
)

month_type_summary
```

```
[169]:
```

	Month	Type	Total_Count
4	May	Slug	293
5	May	Snail	1258
2	Jun	Slug	452
3	Jun	Snail	1650
8	Oct	Slug	35
9	Oct	Snail	131
6	Nov	Slug	116
7	Nov	Snail	2731
0	Dec	Slug	10
1	Dec	Snail	52

```
[170]: # Set up the figure for subplots (2 rows, 1 column)
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(16, 6))

# Plot 1: Line plot for Total Observations per Season
sns.lineplot(data=seasonal_type_summary, x='Season', y='Total_Count',
             ↪hue='Type',
             marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.
             ↪7, ax=ax1)

season_order = seasonal_type_summary['Season'].unique()

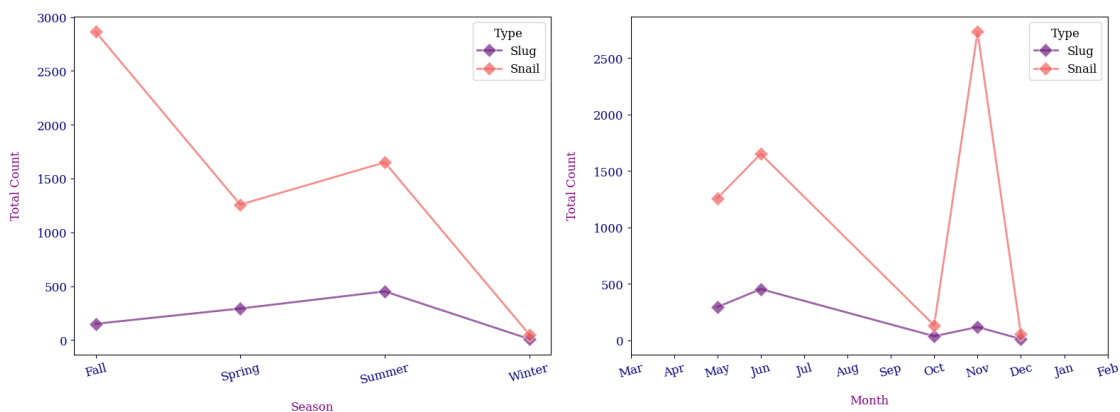
# Customize ax1
ax1.set_xlabel('Season', fontsize=12)
ax1.set_ylabel('Total Count', fontsize=12)
ax1.set_xticks(range(len(season_order))) # Set tick positions
ax1.set_xticklabels(season_order, rotation=15)

# Plot 2: Line plot for Total Observations per Month
sns.lineplot(data=month_type_summary, x='Month', y='Total_Count', hue='Type',
             marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.
             ↪7, ax=ax2)

# Customize ax2
ax2.set_xlabel('Month', fontsize=12)
ax2.set_ylabel('Total Count', fontsize=12)
#ax2.set_title('Total Observations of Slug and Snail per Month', fontsize=14)
ax2.set_xticks(range(len(month_order)))
ax2.set_xticklabels(month_order, rotation=15)

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the combined plots
plt.show()
```



1.1.13 Same plot with Scalling Illusion

```
[171]: # Set up the figure for subplots (2 columns, 1 row)
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(16, 6))

#Change the order of Seasons on X Axis
new_order = ['Winter', 'Spring', 'Summer', 'Fall']
seasonal_type_summary['Season'] = pd.Categorical(
    seasonal_type_summary['Season'], categories=new_order, ordered=True
)

# Plot 1: Line plot for Total Observations per Season with scale illusion
sns.lineplot(data=seasonal_type_summary, x='Season', y='Total_Count',
    hue='Type',
    marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.
    7, ax=ax1)

# Customize ax1 with scale illusion (start y-axis at 500 instead of 0)
ax1.set_ylim(200, seasonal_type_summary['Total_Count'].max() + 100) # Y-axis
    doesn't start at 0
ax1.set_xlabel('Season', fontsize=12)
ax1.set_ylabel('Total Count', fontsize=12)
#ax1.set_title('Total Observations of Slug and Snail per Season (with Scale
    Illusion)', fontsize=14)

#ax1.set_xticks(range(len(new_order))) # Set tick positions
ax1.set_xticklabels(season_order, rotation=15)

# Plot 2: Line plot for Total Observations per Month with scale illusion
sns.lineplot(data=month_type_summary, x='Month', y='Total_Count', hue='Type',
    marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.
    7, ax=ax2)

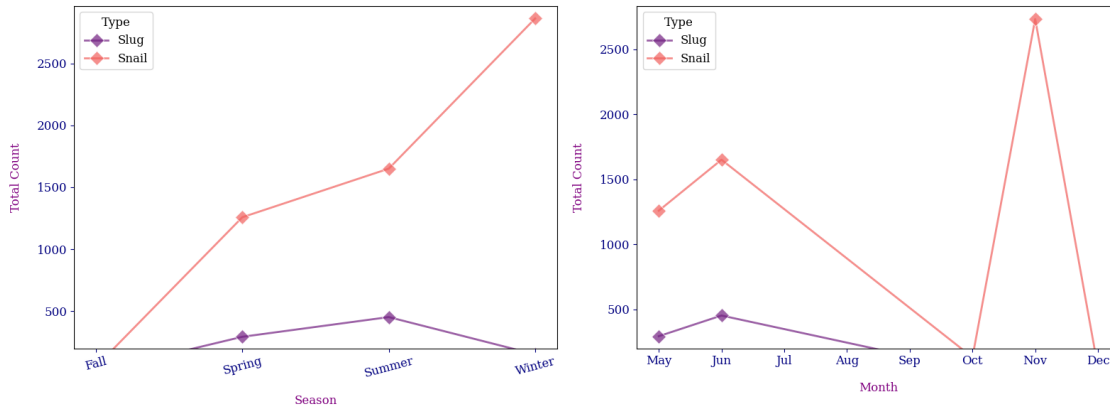
# Customize ax2 with scale illusion (start y-axis at 300 instead of 0)
# X axis are missing some of the months where no observations were recorded
ax2.set_ylim(200, month_type_summary['Total_Count'].max() + 100) # Y-axis
    doesn't start at 0
ax2.set_xlabel('Month', fontsize=12)
ax2.set_ylabel('Total Count', fontsize=12)
#ax2.set_title('Total Observations of Slug and Snail per Month (with Scale
    Illusion)', fontsize=14)

# Adjust layout to avoid overlap
```

```
plt.tight_layout()

# Show the combined plots
plt.show()
```

C:\Users\msyed\AppData\Local\Temp\2\ipykernel_28520\1794555553.py:22:
 UserWarning: set_ticklabels() should only be used with a fixed number of ticks,
 i.e. after set_ticks() or using a FixedLocator.
 ax1.set_xticklabels(season_order, rotation=15)



1.1.14 2nd Plot

```
[172]: type_species_summary[type_species_summary.Total_Count <= 5]
```

```
[172]:
```

	Year	Type	Species_Group	Total_Count
1	2006	Slug	Hesperarion mariae	1
2	2006	Slug	Other Species	3
8	2006	Snail	Haplotrema sp.	2
23	2011	Snail	Allogona townsendiana	3
26	2011	Snail	Euconulus fulvus	2
29	2011	Snail	Megomphix hemphilli	2
49	2012	Snail	Zonitoides arboreus	4

```
[173]: # Adjust overall figure size to make the plots larger
plt.figure(figsize=(14, 12)) # Increase figure size to make it more readable

# Create a FacetGrid to show Slugs and Snails in separate facets, exclude minor
# entries to reduce cluttering
g = sns.FacetGrid(data=type_species_summary[type_species_summary.Total_Count >=
↪ 10],
                  col='Type', height=6, aspect=0.8)
```

```

# Map the line plot onto the FacetGrid, with separate lines for each
↳Species_Group
g.map_dataframe(sns.lineplot, x='Year', y='Total_Count', hue='Species_Group',
↳marker='o',
                    markersize=10, linewidth=3, alpha=0.8)

# Add a legend for the Species_Group and move it outside the plot area
g.add_legend(title="Species", bbox_to_anchor=(1.22, 0.97), loc='upper right')

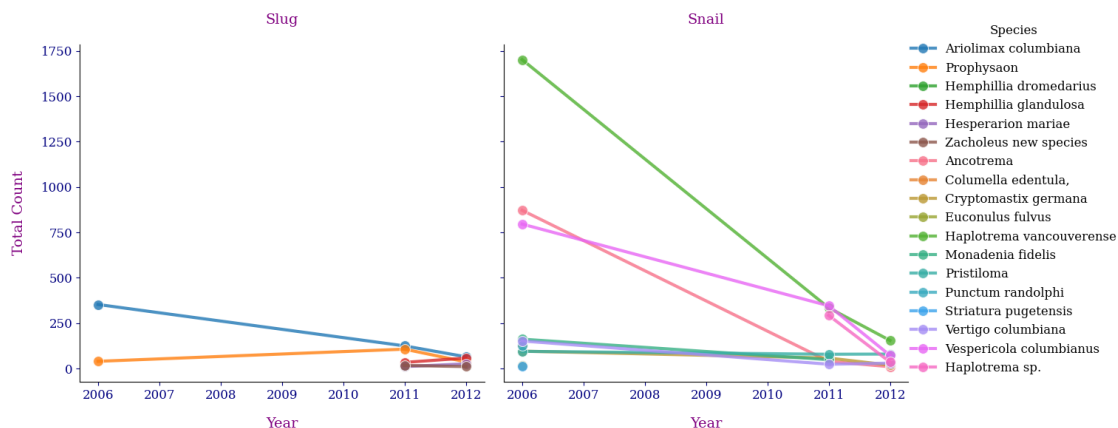
# Adjust axis labels and titles
g.set_axis_labels("Year", "Total_Count", color='purple')
g.set_titles("{col_name}") # Shows Slug or Snail in the column titles

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the plot
plt.show()

```

<Figure size 1400x1200 with 0 Axes>



1.1.15 Same plot with Illusion (Eibbengaus Illusion)

```

[174]: # Adjust overall figure size to make the plots larger
plt.figure(figsize=(14, 12)) # Increase figure size to make it more readable

# Introduce an Ebbinghaus-like illusion by adjusting marker sizes based on a
↳pattern
type_species_summary['Marker_Size'] = np.random.randint(50, 301,
↳size=len(type_species_summary))
#type_species_summary['Marker_Size'] = np.where(type_species_summary['Year'] %
↳2 == 1, 200, 20)

```

```

# Create a FacetGrid to show Slugs and Snails in separate facets
g = sns.FacetGrid(data=type_species_summary[type_species_summary.Total_Count >=
↪10],
                  col='Type', height=6, aspect=0.8)

# Map the line plot onto the FacetGrid, with separate lines for each
↪Species_Group and marker sizes
g.map_dataframe(sns.lineplot, x='Year', y='Total_Count', hue='Species_Group',
                marker='o', linewidth=3, alpha=0.8)
g.map_dataframe(sns.scatterplot, x='Year', y='Total_Count', hue='Species_Group',
                size='Marker_Size', sizes=(50, 300), legend=False, alpha=0.8)

# Add a legend for the Species_Group and move it outside the plot area
g.add_legend(title="Species", bbox_to_anchor=(1.22, 0.97), loc='upper right')

# Adjust axis labels and titles
g.set_axis_labels("Year", "Total Count", color='purple')
g.set_titles("{col_name}") # Shows Slug or Snail in the column titles

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the plot
plt.show()

```

<Figure size 1400x1200 with 0 Axes>



1.1.16 Some other plots variations

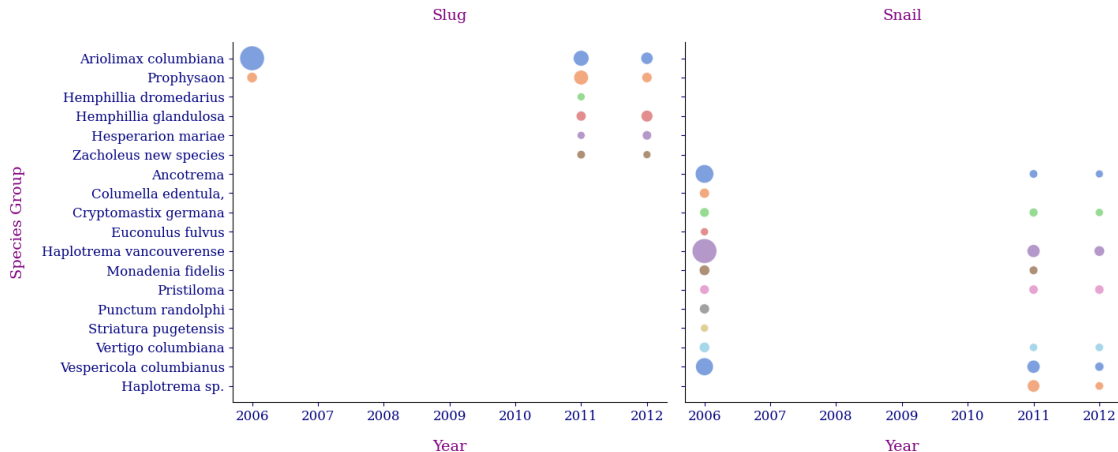
```
[175]: # Create a FacetGrid for the bubble plot, split by 'Type'
g = sns.FacetGrid(data=type_species_summary[type_species_summary.Total_Count >= 10],
                  col='Type', col_wrap=2, height=6, aspect=1.2)

# Map the bubble plot onto the FacetGrid using map_dataframe for more flexibility
g.map_dataframe(sns.scatterplot, x='Year', y='Species_Group',
               size='Total_Count',
               hue='Species_Group', sizes=(50, 500), alpha=0.7,
               palette='muted')

#g.add_legend(title="Species Count Size")
g.set_axis_labels("Year", "Species Group")
g.set_titles("{col_name}")

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the FacetGrid
plt.show()
```



```
[176]: # Create a line plot to show the trend of slug and snail counts over time by month and type
plt.figure(figsize=(10, 5))
sns.lineplot(data=month_type_summary, x='Month', y='Total_Count', hue='Type',
             marker='D', markersize=10, linewidth=2, palette='magma', alpha=0.7)

# Add titles and labels
```

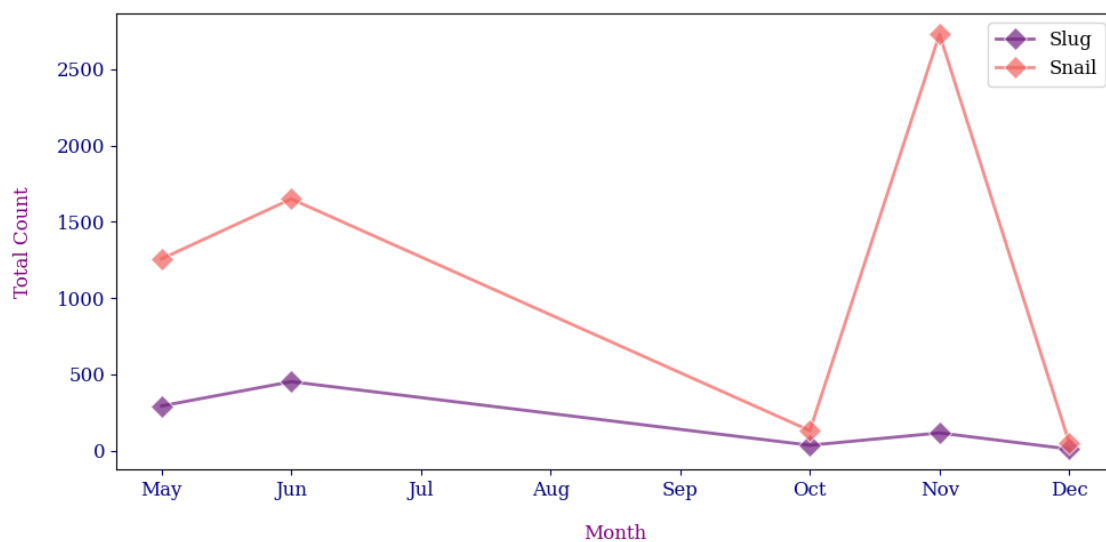
```

plt.title('Total Observations of Slug and Snail by Month', fontsize=16)
plt.xlabel('Month', fontsize=12)
plt.ylabel('Total Count', fontsize=12)
plt.legend(title='')
ax2.set_xticks(range(len(month_order)))
ax2.set_xticklabels(month_order, rotation=15)

# Adjust layout to avoid label overlap
plt.tight_layout()

# Show the plot
plt.show()

```



```

[177]: year_month_type_summary = df_filtered_species.groupby(
        by=['Month Year No', 'Type']
    ).agg(
        Total_Count = ('Count', 'sum')
    ).reset_index()

year_month_type_summary

```

```

[177]:   Month Year No  Type  Total_Count
0    2006-05-01  Slug           231
1    2006-05-01  Snail          1065
2    2006-06-01  Slug           120
3    2006-06-01  Snail          637
4    2006-11-01  Slug            46
5    2006-11-01  Snail         2468
6    2011-06-01  Slug           196

```

7	2011-06-01	Snail	794
8	2011-10-01	Slug	35
9	2011-10-01	Snail	131
10	2011-11-01	Slug	64
11	2011-11-01	Snail	261
12	2011-12-01	Slug	10
13	2011-12-01	Snail	52
14	2012-05-01	Slug	62
15	2012-05-01	Snail	193
16	2012-06-01	Slug	136
17	2012-06-01	Snail	219
18	2012-11-01	Slug	6
19	2012-11-01	Snail	2

```
[178]: df_filtered_species.sort_values('Date', inplace=True)
df_filtered_species.reset_index(drop=True, inplace=True)
df_filtered_species
```

```
[178]:
```

	Date	Year	Month	Year	Month	Month	Year	No	Season	Type	\
0	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
1	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
2	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
3	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
4	2006-05-15	2006	May	2006	May	2006-05-01	Spring	Slug			
...	...	...	...	...	...	...	...	...			
3324	2012-06-19	2012	Jun	2012	Jun	2012-06-01	Summer	Snail			
3325	2012-11-28	2012	Nov	2012	Nov	2012-11-01	Fall	Snail			
3326	2012-11-29	2012	Nov	2012	Nov	2012-11-01	Fall	Slug			
3327	2012-11-29	2012	Nov	2012	Nov	2012-11-01	Fall	Slug			
3328	2012-11-29	2012	Nov	2012	Nov	2012-11-01	Fall	Snail			

	ScientificName	Count	Species_Group
0	Ariolimax columbiana	1	Ariolimax columbiana
1	Ariolimax columbiana	1	Ariolimax columbiana
2	Ariolimax columbiana	1	Ariolimax columbiana
3	Ariolimax columbiana	1	Ariolimax columbiana
4	Ariolimax columbiana	1	Ariolimax columbiana
...	...	...	...
3324	Vertigo columbiana	1	Vertigo columbiana
3325	Haplotrema vancouverense	1	Haplotrema vancouverense
3326	Hemphillia glandulosa	4	Hemphillia glandulosa
3327	Hesperarion mariae	2	Hesperarion mariae
3328	Haplotrema vancouverense	1	Haplotrema vancouverense

[3329 rows x 10 columns]

```
[179]: # Aggregate the data by Year, Season, and Type to get the total count
trend_data = df_filtered_species.groupby(['Month Year No', 'Month Year', 'Type']).agg(
    Total_Count=('Count', 'sum')
).reset_index()

trend_data
```

```
[179]:
```

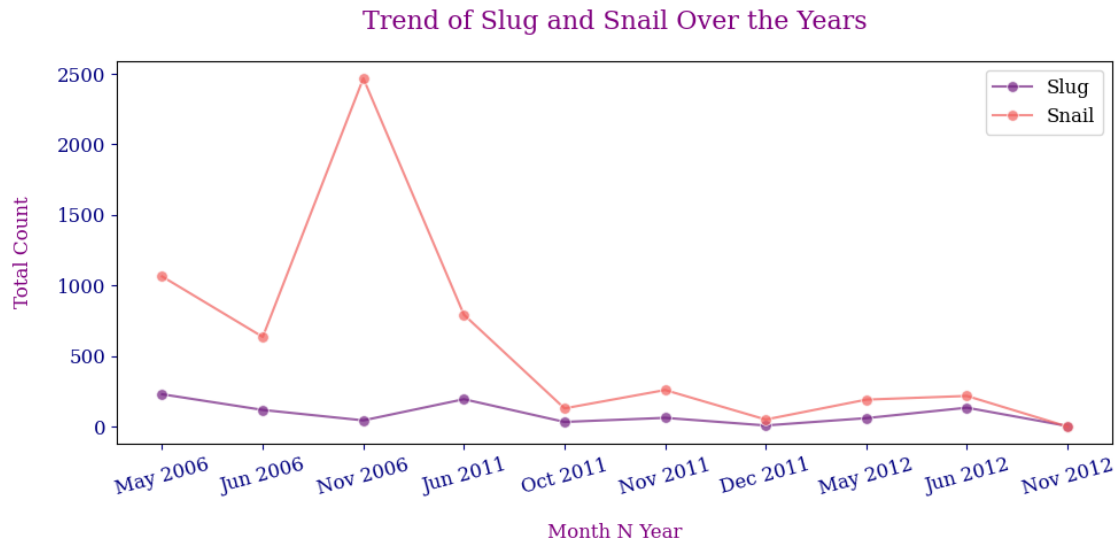
	Month Year No	Month Year	Type	Total_Count
0	2006-05-01	May 2006	Slug	231
1	2006-05-01	May 2006	Snail	1065
2	2006-06-01	Jun 2006	Slug	120
3	2006-06-01	Jun 2006	Snail	637
4	2006-11-01	Nov 2006	Slug	46
5	2006-11-01	Nov 2006	Snail	2468
6	2011-06-01	Jun 2011	Slug	196
7	2011-06-01	Jun 2011	Snail	794
8	2011-10-01	Oct 2011	Slug	35
9	2011-10-01	Oct 2011	Snail	131
10	2011-11-01	Nov 2011	Slug	64
11	2011-11-01	Nov 2011	Snail	261
12	2011-12-01	Dec 2011	Slug	10
13	2011-12-01	Dec 2011	Snail	52
14	2012-05-01	May 2012	Slug	62
15	2012-05-01	May 2012	Snail	193
16	2012-06-01	Jun 2012	Slug	136
17	2012-06-01	Jun 2012	Snail	219
18	2012-11-01	Nov 2012	Slug	6
19	2012-11-01	Nov 2012	Snail	2

```
[180]: # Create a line plot to show the trend of slug and snail counts over time by
        ↪season and type
plt.figure(figsize=(10, 5))
sns.lineplot(data=trend_data, x='Month Year', y='Total_Count', hue='Type',
             marker='o', markersize=7, palette='magma', alpha=0.7)

# Add titles and labels
plt.xticks(rotation=15)
plt.title('Trend of Slug and Snail Over the Years', fontsize=16)
plt.xlabel('Month N Year', fontsize=12)
plt.ylabel('Total Count', fontsize=12)
plt.legend(title='')

plt.tight_layout()

# Show the plot
plt.show()
```



```
[181]: # Create a bar plot
fig, ax = plt.subplots(figsize=(8, 5))

bp = sns.barplot(data=seasonal_type_summary,
                 x='Season',
                 y='Total_Count',
                 hue='Type',
                 palette='pastel'
                )

for i, bar in enumerate(bp.patches):
    if bar.get_height() <= 0: continue      # Skip 0 entries
    ax.text(
        bar.get_x() + bar.get_width() / 2, # X position of the text (center of
        ↪ the bar)
        bar.get_height() + 0.25,           # Y position of the text (height of
        ↪ the bar)
        f'{int(bar.get_height())}',         # The text (formatted as integer
        ↪ with commas)
        ha='center', va='bottom',          # Text alignment: horizontal
        ↪ 'center', vertical 'bottom'
        fontsize=8, color='black',         # Text font properties
        fontfamily='serif'
    )

# Add titles and labels
plt.title('Total Count of Slugs and Snails per Season')
plt.xlabel('Season')
```

```
plt.ylabel('Total Count')
plt.legend(title='')

plt.tight_layout()

# Show the plot
plt.show()
```

